Обработка на изключения

Калин Георгиев

19 май 2026 г.

Механизъм на изключенията

- Алтернатива на “нормалния резултат”
- Само за ситуации, които биха нарушили “нормалната” програмна логика
- Позволяват “възстановяване от грешки”

Какво е “Грешка”? Защо стават грешки?

Елементи на изключенията

- Възникване: `throw`
- Следене: `try`
- Прихващане: `catch`

throw

- Повече от един `throw` във функция
- Тип на изключението

catch

- Тип на изключението
- Различни `catch` клаузи за различните типове изключения
- std::exception

Жизнен цикъл на обектите при изключения

- `Stack unwinding`
- Изключения в конструктори
  - RAII: Притежанието на ресурс е инварианта на класа
- Изключения в деструктори
  - Изключение по време на обработка на изключение
  - Прехвърляне на отговорността на потребителя (напр. std::fstream::close)

Exception Safety

Какво е Exception Safety?

- Декларация за поедение относно изключения
- Три нива на гаранция (по Abrahams)
  - Basic exception safety
  - Strong exception safety
  - No-throw exception safety
- Отражение върху за RAII инварианта

Unsafe code

- Състоянието на обекта може да се инвалидира
- Унищожение на обекта води до изтичане на памет
- Няма начин да разберем дали операцията е успешна
- Създават се условия за катастрофални грешки

```
class Vector {
private:
    int* data;
    size_t size;
public:
    void push_back(int x) {
        data = realloc(data, (size+1)*sizeof(int));
        // ralloc may be nullpt
        data[size++] = x;
    }
};
```

Basic Exception Safety

- Гарантирано няма изтичане на ресурси
- Сътоянието на обекта може да стане “нелогично”, но не и катастрофално
- “Може да се счупи”, но няма “да счупи програмата”

```
class Vector {
private:
    int* data;
    size_t size;
public:
    void push_back(int x) {
        int *save = data;
        data = realloc(data, (size+1)*sizeof(int));
        // ralloc may be nullpt
        if (data == nullptr) {
            delete[] save;  //!!!
            size = 0;       //!!!
            throw std::bad_alloc();
        } else {
            data[size++] = x;
        }
    }
```

Strong Exception Safety

- Операцията невинаги е успешна, но ако не е, състоянието на обекта не се променя
- "Всичко или нищо"

```
class Vector {
private:
    int* data;
    size_t size;
public:
    void push_back(int x) {
        int *save = data;
        data = realloc(data, (size+1)*sizeof(int));
        // ralloc may be nullpt
        if (data == nullptr) {
            data = save;  //!!!
            throw std::bad_alloc();
        }

        data[size++] = x;
    }
};
```

Nothrow Exception Safety

- Операцията винаги е успешна
- Деструкторите ТРЯБВА да са no-throw

```
class MyVector {
public:
    void swap(MyVector& other) noexcept {
        std::swap(data, other.data);
        std::swap(size, other.size);
    }

};
```

Алтернативи във функционален стил

- `std::pair<T,bool>`
- `std::optional<T>`

Благодаря за вниманието!